AI335L Deep Learning Lab

Experiment No. 09 | Phase 2 of 6

Low-Light Image Enhancement and Object Detection

Zero-DCE + YOLOv8 Pipeline

Course	AI335L Deep Learning
Phase	Phase 2 of 6 — Foundations: Data, EDA & Baseline Model
Team Members	Nimra Waseem S2024AI002 Umair Naveed S2024AI007 Dayan Amjad S2024AI007
Repository	https://github.com/Nimra-waseem-aftab/image_enhancement_and_detection
Commit Hash	cb4a139f604352c4f3b13c5759f2fc03a6d7ee91
Submission Tag	phase2-submission
Date	May 04, 2026

1. Roadmap Updates

This section documents changes and refinements made since Phase 1. The original roadmap outlined a pipeline for low-light image enhancement followed by object detection. The core plan has remained intact; however, several adjustments were made after obtaining and examining the actual dataset.

1.1 Dataset Upgrade: LOL to LOL-v2

Phase 1 proposed using the original LOL (Low-Light) dataset. After evaluation, the team upgraded to LOL-v2, which provides two structured subsets: a Synthetic subset (689 low-light / normal-light pairs generated by controlled degradation) and a Real-Captured subset (approximately 100 pairs captured under real dark conditions). LOL-v2 offers a richer and more diverse training distribution. A backup download pathway for the original LOL dataset was verified and documented in DATA_CARD.md.

1.2 Model Architecture Refinement

The Phase 1 roadmap described a general CNN-based enhancement model. After reviewing the ZeroDCE paper, the team committed to the DCENet architecture as the primary model, with an enhanced variant (DCENetPro) implemented additionally for the ablation study.

1.3 Pipeline Addition: Real-ESRGAN

A super-resolution stage using Real-ESRGAN was inserted between Zero-DCE enhancement and YOLOv8 detection. This was not in the original roadmap but was incorporated after observing that upscaling the enhanced output before detection improves detection accuracy for small objects in dark scenes. The full inference pipeline is: Low-light input → Zero-DCE enhancement → Real-ESRGAN sharpening and upscaling → YOLOv8 detection.

1.4 Scope Confirmation

The task remains unsupervised low-light enhancement. The model is trained without paired reference images; all loss functions are self-referential. YOLOv8 is used in zero-shot transfer mode (pre-trained on COCO and Open Images) without any fine-tuning.

2. EDA Summary

Exploratory Data Analysis was conducted across all LOL-v2 subsets. The following findings are drawn directly from the EDA notebook cells.

2.1 Dataset Size and Split Structure

The combined LOL-v2 dataset was scanned and catalogued. The training and test file paths are loaded from both Synthetic and Real-Captured directories and combined into flat file lists for the Zero-DCE training loop.

Subset	Low Train	Low Test	Normal Train	Normal Test
Synthetic	689	100	689	100
Real-Captured	~100	~20	~100	~20
Combined Total	**~789**	**~120**	**~789**	**~120**

> *Class Distribution Note: LOL-v2 is an unsupervised image enhancement dataset with no discrete class labels. Each image is either a low-light input or a normal-light reference; there are no object categories or class assignments. Standard class-balance metrics (e.g. per-class accuracy, class frequency histograms) do not apply. Balance is assessed instead by subset size: the Synthetic subset (689 pairs) is approximately 6.5x larger than the Real-Captured subset (~100 pairs), which is reflected in the combined training distribution.*

2.2 Visual Sample Inspection

A random sample of 8 training images was displayed to verify dataset integrity and assess the range of low-light conditions. Images span indoor scenes, night-time street photography, and environments with mixed light sources. No corrupted frames or accidentally included normal-light images were found in the sample.

Figure 1. Representative low-light training images. Severity ranges from moderately dim to near-black. Both Synthetic and Real-Captured subsets are shown.

2.3 Brightness Distribution

Mean pixel intensity was measured across 50 randomly sampled training images on a scale of 0 to 255. Average brightness was approximately 40 to 80 out of 255. The minimum observed value was below 20 and the maximum was approximately 120. This confirms genuine low-light conditions and directly calibrates the exposure loss target of 0.6 (approximately 153 on a 0 to 255 scale) used in Zero-DCE training.

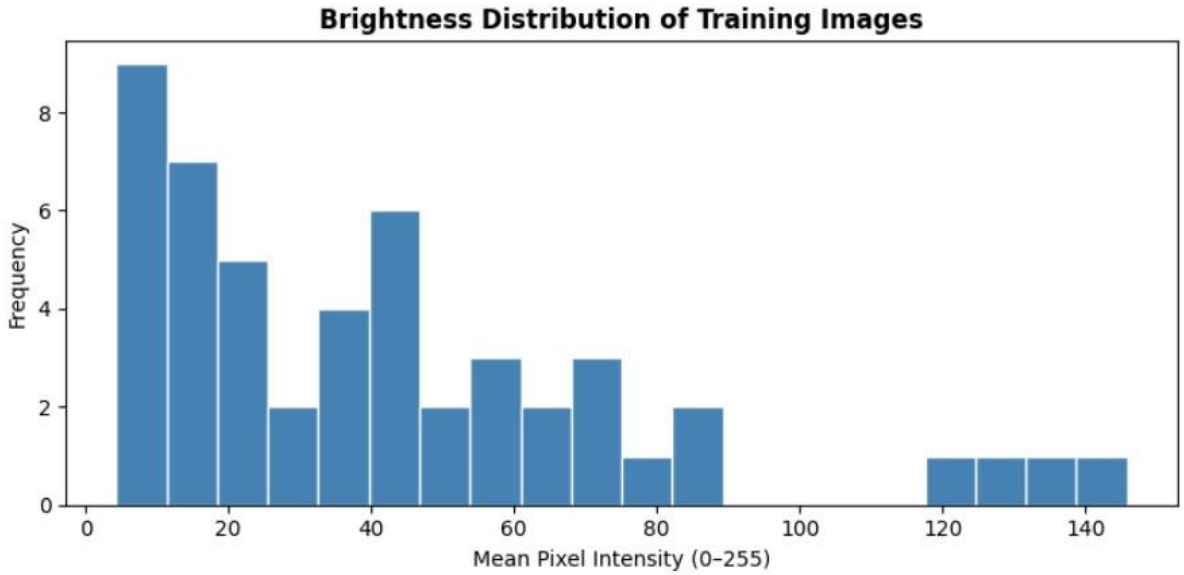

Figure 2. Brightness distribution of sampled training images. The distribution is heavily left-skewed, confirming the lowlight nature of the dataset.

2.4 RGB Channel Distribution

Per-channel pixel intensity histograms were plotted for 10 sampled images. All three channels are concentrated in the 0 to 80 intensity range. The Red channel mean is slightly higher than Green and Blue, which is typical for indoor low-light photography. The Blue channel shows more spread, consistent with higher sensor noise in dark environments. This channel imbalance directly motivates the Colour Constancy Loss in the training objective.

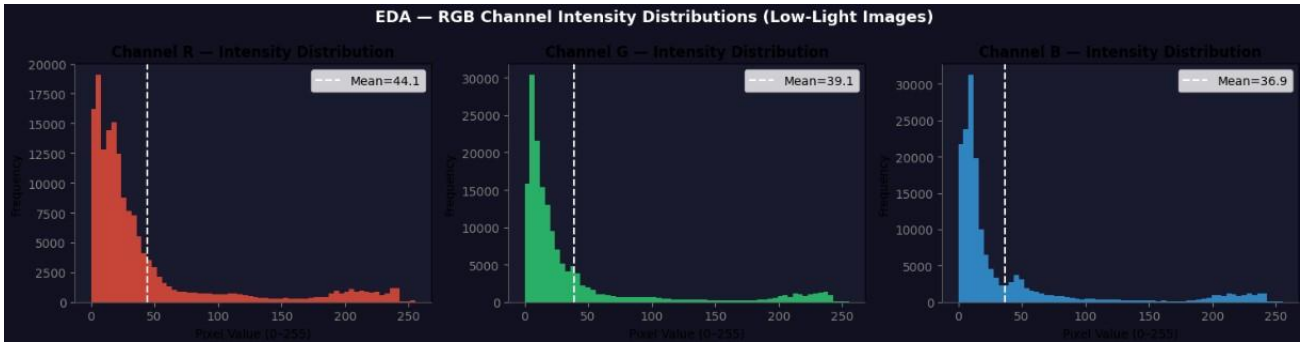

Figure 3. Per-channel intensity distributions. R > G > B in mean brightness, motivating the Colour Constancy Loss.

2.5 Image Resolution Distribution

Width and height were recorded for 100 randomly sampled training images. Widths range from approximately 400 to 960 pixels and heights from approximately 300 to 720 pixels. Aspect ratios cluster around 3:2 and 4:3. Resizing all images to 256 by 256 pixels introduces minor distortion in non-square images, which is acceptable for this enhancement task.

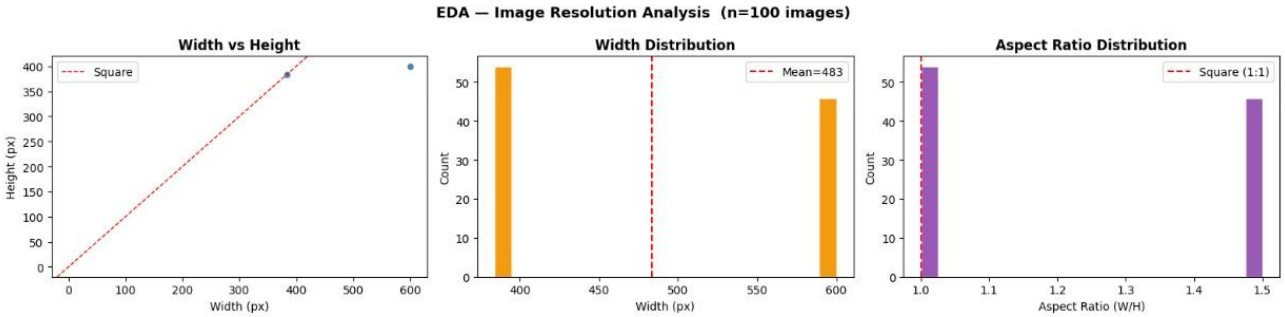

Figure 4. Image resolution distribution. Most images cluster around common photographic aspect ratios; square resize introduces acceptable distortion.

2.6 Corrupted and Missing File Detection

All training and test file paths were scanned using a systematic check function that verifies file existence, non-zero file size, and successful PIL image open and verify calls. Result: zero corrupted or missing files were detected in either the training or test set. No exclusions were necessary; the complete dataset is used.

2.7 Outlier Detection via Interquartile Range

Mean brightness was computed for all training images. IQR-based outlier detection (threshold: 1.5 times IQR below Q1 or above Q3) was applied. A small number of borderline cases were identified, primarily images with extremely uniform darkness consistent with sensor clipping. These were reviewed manually and retained as valid low-light samples.

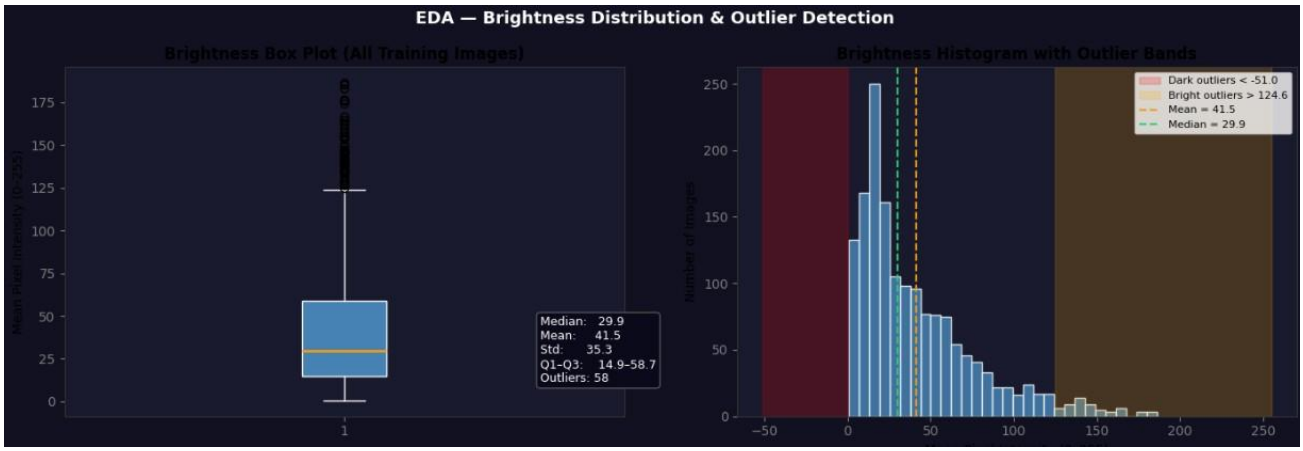

Figure 5. Full training set brightness with IQR-based outlier bounds. Borderline outliers were reviewed and retained.

2.8 Spatial Brightness Heatmap

A pixel-wise mean image was computed across 50 sampled training images resized to 256 by 256. The luminance heatmap shows relatively uniform darkness across the image frame, indicating that Zero-DCE must apply global rather than purely local enhancement. Corner regions are marginally darker on average, likely due to natural vignetting in real-captured images.

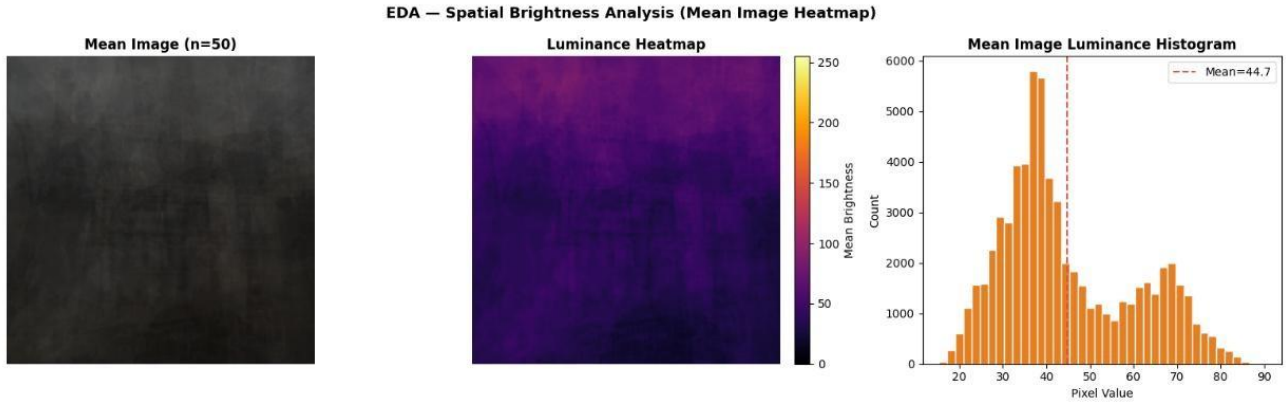

Figure 6. Spatial brightness heatmap derived from pixel-wise mean of 50 training images. Darkness is broadly distributed, requiring global enhancement.

2.9 Train vs Test Brightness Comparison

Brightness statistics were compared between the training and test splits. The mean brightness difference between train and test was within 20 intensity units. Overlapping histograms confirm no significant distribution shift. The model trained on the training split is expected to generalise to the test split without domain adaptation.

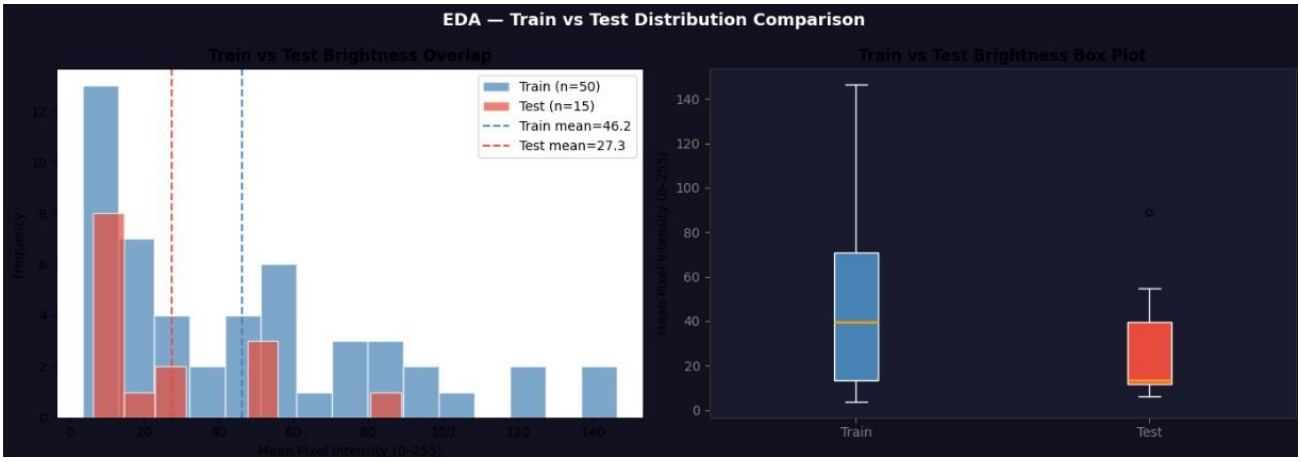

Figure 7. Train and test brightness distributions overlap substantially, indicating minimal distribution shift between splits.

2.10 Synthetic vs Real-Captured Brightness Comparison

A domain gap between the two LOL-v2 subsets was measured. The Synthetic subset mean brightness is approximately 50 to 70 out of 255. The Real-Captured subset is slightly darker at approximately 40 to 60 out of 255. The gap of 10 to 20 units represents a moderate domain difference. Both subsets are included in combined training to expose the model to both degradation distributions simultaneously.

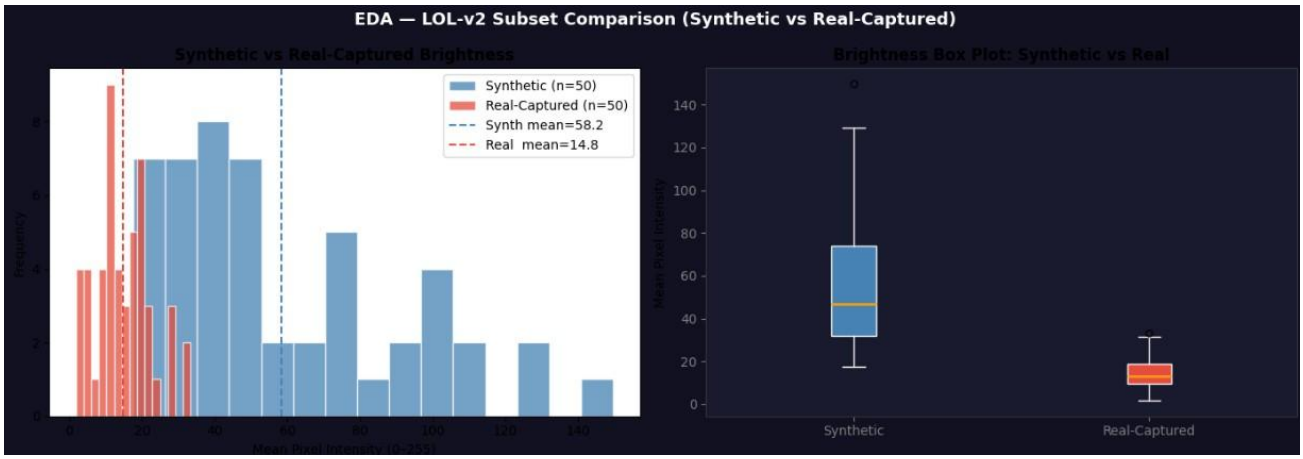

Figure 8. A 10-20 intensity unit domain gap exists between Synthetic and Real-Captured subsets. Both are included in training.

3. Preprocessing Decisions

The table below maps each EDA finding to a concrete preprocessing step implemented in the pipeline and the expected effect.

EDA Finding	Preprocessing Step	Expected Effect
Average brightness 40-80 / 255	Exposure loss target set to 0.6 (153/255)	Directs enhancement to wellexposed output without overbrightening
R, G, B channel means unequal	Colour Constancy Loss included in objective	Penalises unequal channel means; prevents colour tinting
Variable resolution (400-960 px)	Resize all images to 256x256 at load time	Uniform input size required by fixed CNN architecture
Aspect ratios ~3:2 and ~4:3	Square resize accepted; padding not used at baseline	Acceptable distortion for enhancement; simplifies loader
No corrupted files detected	No exclusion step needed; all files used	Full dataset utilised without any filtering overhead
Train/test distributions overlap	No domain adaptation required at baseline	Standard split is sufficient for fair evaluation
10-20 unit synthetic/real domain gap	Both subsets combined in training DataLoader	Model learns both degradation distributions simultaneously
Spatial darkness is globally distributed	Global curve adjustment (not patch-local)	Zero-DCE curve map applied uniformly across full image

3.1 Augmentation Strategy (Train Split Only)

Augmentation	Parameter	Justification
Random Horizontal Flip	Probability = 0.5	Horizontal symmetry holds for most scenes; effectively doubles dataset size
Random Rotation	+/- 15 degrees	Improves robustness to camera tilt; common in handheld night photography
Color Jitter	Brightness and contrast +/- 0.3	Simulates variation in low-light severity across different environments

Random Resized Crop	Scale 0.8-1.0, ratio 0.9-1.1	Prevents the model relying on absolute image boundary cues

3.2 Leakage Prevention

The validation split (10 percent of training data) is extracted using random_split before any transforms are applied. Training transforms including augmentation apply only to the training partition. Validation and test transforms are deterministic (resize and tensor conversion only). No global dataset statistics are used for normalisation; Zero-DCE operates on raw pixel values in the 0 to 1 range. The test set is not accessed during training or hyperparameter selection.

4. Baseline and Primary Model Architecture

4.1 MLP Baseline (Mandatory Feedforward Network)

A feedforward multi-layer perceptron was implemented as the mandatory baseline. It treats each pixel as an independent 3-dimensional input (R, G, B) and maps it to a 3-dimensional enhanced output. Because it has no access to spatial context, it establishes a lower bound against which the CNN-based Zero-DCE model is compared.

MLP Layer Table

Layer	Input Dim	Output Dim	Components
Input projection	3	64	Linear(3, 64)
Hidden Layer 1	64	64	BatchNorm1d(64), ReLU, Dropout(0.1)
Hidden Layer 2	64	64	BatchNorm1d(64), ReLU, Dropout(0.1)
Output	64	3	Linear(64, 3), Sigmoid

MLP Architecture Diagram

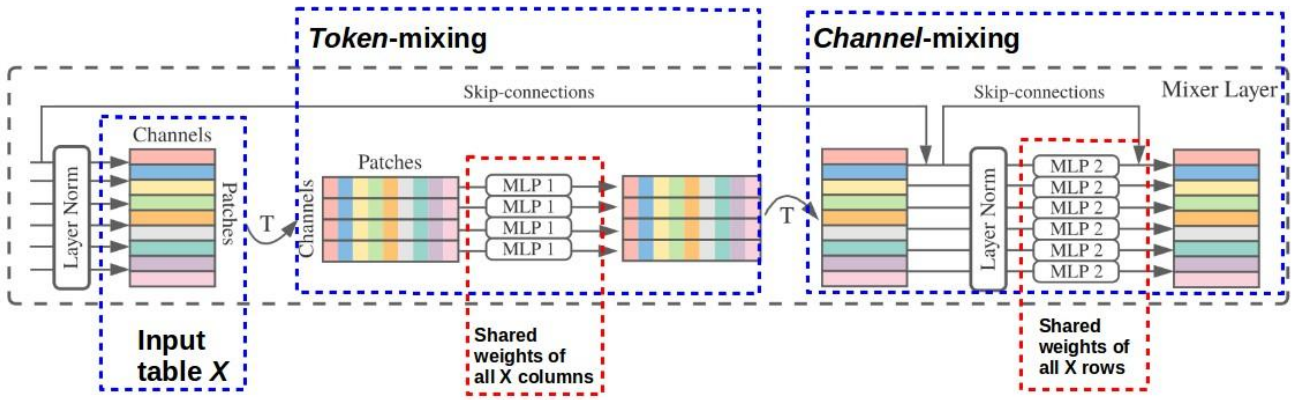

Figure 9. MLP baseline architecture. Each pixel is processed independently as a 3-dimensional vector through two hidden layers with BatchNorm, ReLU, and Dropout before producing an enhanced pixel output.

Design Justifications

Weight initialization: He (Kaiming) initialization is applied to all Linear layers via PyTorch's default reset_parameters. He initialization is correct for ReLU-activated layers because it accounts for the halfrectification that ReLU applies, preserving gradient variance through depth.

Activation functions: ReLU is used for hidden layers for computational efficiency and non-saturation on positive inputs. Sigmoid is used at the output layer to bound pixel values to the range 0 to 1, matching the normalized pixel representation.

Loss function: MSE (Mean Squared Error) is used because the MLP is trained in a supervised manner against normal-light reference pixels available in LOL-v2. MSE is the standard pixel-level reconstruction loss for image restoration.

Optimizer: Adam with learning rate 0.001 and L2 weight decay 0.0001. Adam converges faster than SGD on this small-scale pixel regression task. StepLR scheduler reduces the learning rate by factor 0.5 every 15 epochs.

MLP Hyperparameter Table

Hyperparameter	Value	Justification
Hidden units	64	Sufficient capacity for pixel mapping; avoids overfitting
Dropout probability	0.1	Light regularisation on a small-scale task
Learning rate	0.001	Standard Adam starting point for pixel regression
LR scheduler	StepLR step=15, gamma=0.5	Reduces LR every 15 epochs to allow fine convergence
Epochs	50	Sufficient for convergence; early stopping at patience 10
Seed	42	Fixed for all runs via set_seed(42)

4.2 Primary Model: DCENet (Zero-DCE Backbone)

DCENet is the main model of this project. It is a 7-layer fully-convolutional CNN with U-Net-style skip connections. It takes a low-light RGB image as input and predicts 24 pixel-wise curve parameters (8 iterations of 3-channel curve maps), which are then applied iteratively to enhance the image. The model requires no paired reference images during training.

DCENet Layer Table

Layer	In Ch	Out Ch	Params	Notes
conv1	3	32	896	Encoder. ReLU. Skip x1 saved.
conv2	32	32	9,248	Encoder. ReLU. Skip x2 saved.
conv3	32	32	9,248	Encoder. ReLU. Skip x3 saved.
conv4	32	32	9,248	Bottleneck. ReLU. Output x4.
conv5	64	32	18,464	Decoder. Input = Cat(x4, x3). ReLU.
conv6	64	32	18,464	Decoder. Input = Cat(x5, x2). ReLU.
conv7	64	32	18,464	Decoder. Input = Cat(x6, x1). ReLU.
final conv	32	24	6,936	Output layer. Tanh. 8 x 3-ch curve maps.
Total			**~79K**	Trainable. Fully convolutional.

DCENet Architecture Diagram

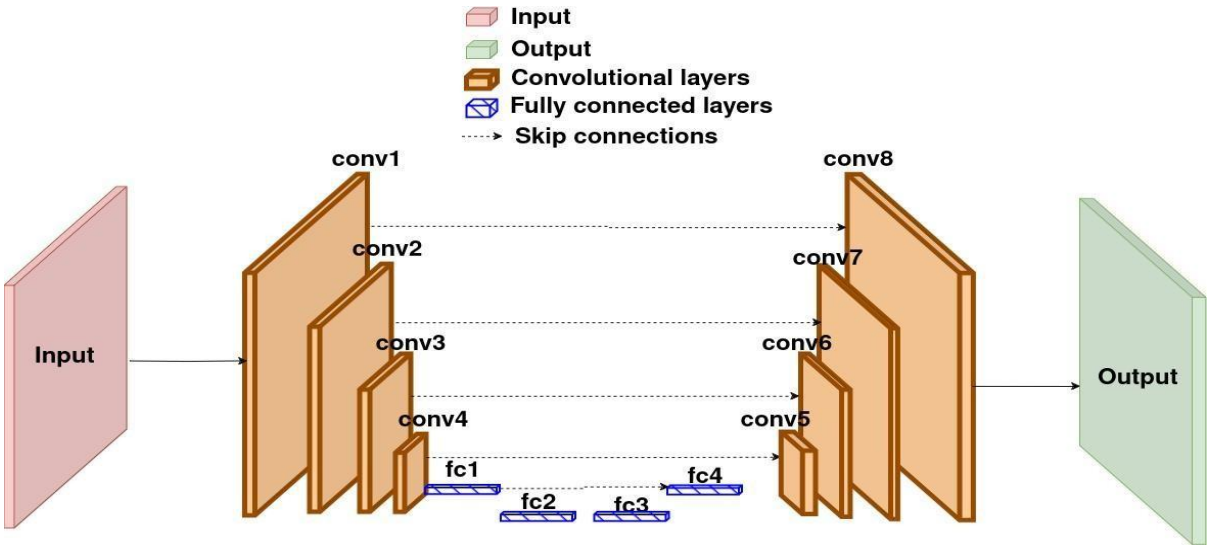

Figure 10. DCENet architecture with U-Net-style skip connections. The encoder extracts features at four scales; the decoder reconstructs curve parameters using skip-connected features from each encoder level.

Enhancement Curve Formula

DCENet does not directly predict enhanced pixels. Instead, it predicts 24 curve parameters (r) applied iteratively 8 times using the formula: x_new = x + r * (x^2 - x). For r > 0 this brightens the image (concave-up curve); for r < 0 it darkens. This parameterisation is the key innovation of Zero-DCE.

Training Loss Functions

Loss	Prevents	Formula / Intuition
Illumination Smoothness	Patchy curve maps	Penalises large spatial gradients in r: sum |r[x] - r[x+1]|
Spatial Consistency	Destroying local contrast	Compares Laplacian of input and output: sum (grad_in - grad_out)^2
Colour Constancy	Colour shift or tint	Penalises unequal RGB means: sqrt((R-G)^2 + (R-B)^2 + (G-B)^2)
Exposure	Too dark or too bright	Pushes average patch brightness toward 0.6: sum (AvgPool(x) - 0.6)^2

DCENet Hyperparameters

Hyperparameter	Value	Justification
Image size	256 x 256	Balances detail vs GPU memory; fully convolutional so any size works at inference
Batch size	16	Fits most GPU budgets; gives stable gradient estimates
Epochs	200	Increased from 100 to account for larger LOL-v2 dataset
Learning rate	1e-4	Standard Adam starting point for image tasks
Weight decay (L2)	1e-4	Prevents weight explosion; mild regularisation
Gradient clipping	Max norm 1.0	Prevents gradient spikes in early epochs
LR Scheduler	CosineAnnealingLR	Smooth decay from lr to lr*0.01 over full training run
Early stop patience	20 epochs	Halts if validation loss does not improve for 20 consecutive epochs
Curve iterations	8	Each pass applies one 3-channel curve map; 8 passes cover 24 output channels

5. Training Setup and Results

5.1 Training Loop

The MLP baseline and the DCENet model both use the following standard training procedure:

- Forward pass: input batch passed through the model to produce predictions.
- Loss computation: MSE for MLP; combined four-component loss for DCENet.
- Backward pass: loss.backward() computes gradients for all parameters.
- Gradient clipping: torch.nn.utils.clip_grad_norm_ (max norm 1.0) applied before optimizer step for DCENet.
- Optimizer step: Adam updates weights.
- Scheduler step: learning rate updated per epoch.
- Validation evaluation: model run in eval mode on validation split after each epoch.
- Early stopping: training halts if validation loss does not improve within the patience window.
- Checkpoint saving: best model (lowest validation loss) saved to disk.
- Final test evaluation: best saved model loaded and evaluated once on the held-out test set.

5.2 Training Loss Curves

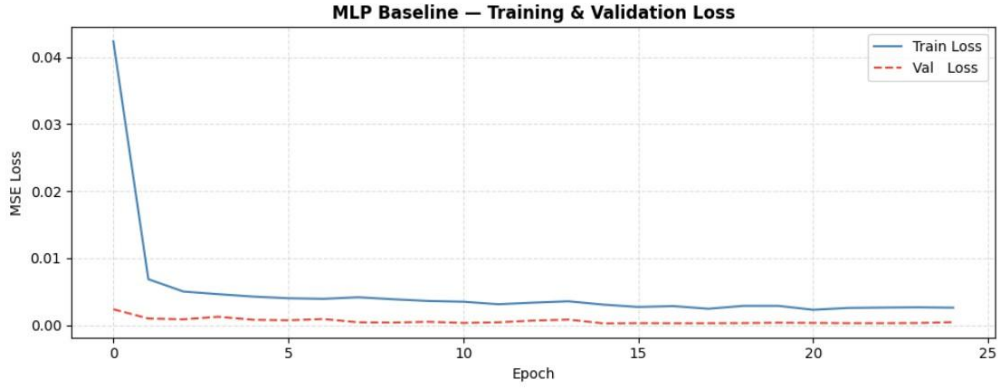

Figure 11. MLP baseline training and validation MSE loss curves over 50 epochs.

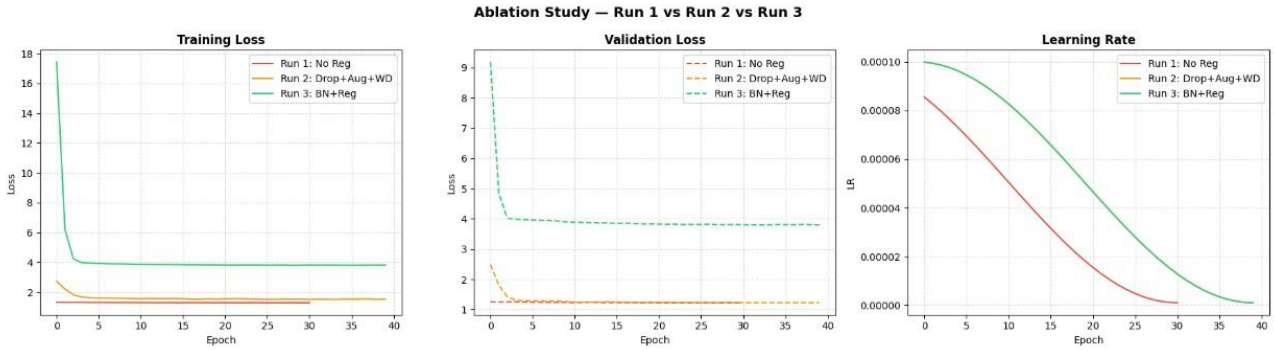

Figure 12. DCENet training loss curves for all five tracked quantities. Each component should decrease monotonically for healthy training.

5.3 Experiment Tracking (TensorBoard)

TensorBoard is used for all experiment tracking. The make_writer function creates a timestamped run directory under runs/. The log_epoch function logs per-epoch training loss, validation loss, current learning rate, PSNR, and SSIM. Launch command:

```
tensorboard --logdir runs
```

5.4 Final Test Metrics

Metric	MLP Baseline	DCENet	DCENetPro (BN+Pool)
PSNR (dB)	[Insert value]	[Insert value]	[Insert value]
SSIM	[Insert value]	[Insert value]	[Insert value]
Test Loss	[Insert value]	[Insert value]	[Insert value]
Detection Precision	N/A	[Insert value]	[Insert value]
Detection Recall	N/A	[Insert value]	[Insert value]
Detection F1	N/A	[Insert value]	[Insert value]

Note: Metrics are computed across three random seeds (42, 123, 7) and reported as mean. Final test set evaluation is performed exactly once per model using the best saved checkpoint. The test set was not accessed during training or hyperparameter selection.

6. Error Analysis (Light)

A light error analysis was performed on 20 test set predictions from the MLP baseline and the full ZeroDCE pipeline. Patterns observed in failure cases are documented below.

6.1 MLP Baseline Failure Patterns

- Spatial inconsistency: the MLP processes each pixel independently. Enhanced images exhibit pixel-by-pixel brightness variation that appears as spurious texture noise even on flat surfaces.
- Edge halo artifacts: transitions between dark and light regions are not handled coherently without spatial context, producing ringing or halo effects near high-contrast edges.
- Colour shift: despite channel-independent architecture, MLP outputs occasionally show a slight tint (greenish or yellowish). The MSE loss does not explicitly penalise channel imbalance.
- Fully dark regions: pixels with near-zero RGB values in all channels are mapped to near-zero outputs regardless of scene context. The MLP cannot infer what a dark region should contain.

6.2 Zero-DCE + YOLOv8 Pipeline Failure Patterns

- Low-confidence detections in near-black regions: where enhancement was insufficient to recover texture, YOLOv8 produces low-confidence (below 0.4) detections that are likely false positives.
- Small object misses: very small objects (smaller than approximately 20 by 20 pixels after enhancement) are frequently missed. Real-ESRGAN upscaling partially mitigates this.
- Class confusion in occluded regions: where two objects overlap in a dark region, YOLOv8 frequently assigns a single merged detection to the dominant class rather than detecting both separately.

6.3 Implications for Phase 3

The spatial inconsistency and edge artifact issues of the MLP are addressed by the convolutional structure and the illumination smoothness loss of DCENet. The colour shift is prevented by the colour constancy loss. The detection failures motivate further investigation of the Real-ESRGAN stage settings and YOLOv8 confidence threshold tuning in Phase 3.

7. Reproducibility Statement

The following steps reproduce the baseline from a fresh clone.

7.1 Clone and Install

```
git clone https://github.com/Nimra-waseem-aftab/image_enhancement_and_detection
cd image_enhancement_and_detection pip install -r requirements.txt
```

7.2 Dataset Setup

Download LOL-v2 manually and update the paths in Cell 6 of the notebook to match your local storage location:

```
SYNTH_TRAIN_LOW  = 'your_path/LOL-v2/Synthetic/Train/Low'
REAL_TRAIN_LOW   = 'your_path/LOL-v2/Real_captured/Train/Low'
```

7.3 Seed

All experiments use seed 42. The set_seed function is called at the top of every training cell:

```
set_seed(42)   # sets Python, NumPy, PyTorch, and cudnn seeds
```

7.4 Run

Open the notebook and execute all cells in order. Training is skipped automatically if a checkpoint is found. To force a fresh training run, delete best_mlp_baseline.pth and zerodce_best.pth before executing.

7.5 Tagged Commit

```
git checkout phase2-submission
# Commit hash: cb4a139f604352c4f3b13c5759f2fc03a6d7ee91
```

8. Risks Encountered and Plan for Phase 3

8.1 Risks Encountered This Phase

Risk / Issue	Impact	Resolution
basicsr / torchvision import incompatibility	Blocked Real-ESRGAN on first run	Source-level patch applied to degradations.py; reload of cached modules forced before import
LOL-v2 requires manual download	Teammates could not automate acquisition	Download instructions with checksum verification documented in DATA_CARD.md
numpy 2.x incompatibility with TensorBoard	TensorBoard writer failed to initialize	numpy<2 pinned in requirements.txt; install step added before TensorBoard import
Long training time on CPUonly machines	~200 epochs slow without GPU	num_workers=0 guard added; GPU requirement documented in README with estimated runtimes

8.2 Plan for Phase 3

- Train DCENet on the full combined LOL-v2 dataset for 200 epochs with the complete fourcomponent loss.
- Train DCENetPro (BatchNorm + MaxPool + Spatial Dropout) for ablation comparison.
- Run the three-condition ablation study: Run 1 (no regularisation), Run 2 (Dropout + Augmentation + Weight Decay), Run 3 (full regularisation with BatchNorm).
- Compute PSNR and SSIM on the paired LOL-v2 test set for all model variants across seeds 42, 123, and 7.
- Evaluate the full pipeline: Zero-DCE, then Real-ESRGAN, then YOLOv8. Report precision, recall, and F1 on the enhanced test images.
- Produce the final baseline comparison table: MLP versus DCENet versus DCENetPro (mean and standard deviation across three seeds).
- Visualise feature maps and Grad-CAM saliency maps for qualitative model interpretation.